

The background of the slide is a dark blue gradient. It is filled with numerous thin, vertical lines of varying heights and colors, including green, blue, and purple, creating a digital or data-like aesthetic.

Recursividad

INFO II

Profesor: Ing. Federico Weber

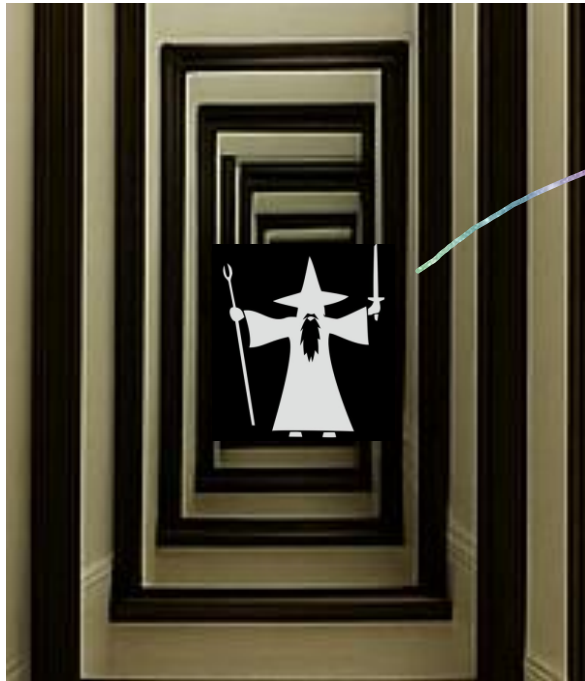
Recursividad

- La **recursividad** es una técnica de programación en la que una función se llama a sí misma para resolver un problema.
- En otras palabras, una solución recursiva se construye definiendo el problema en términos de subproblemas del mismo tipo más pequeños.
- Formalmente, “se dice que un proceso es recursivo si se resuelve llamándose a sí mismo”.



Recursividad

- Para que este proceso tenga un fin correcto (y no se vuelva infinito), **debe existir al menos un caso base** (una condición que no hace más llamadas recursivas) y cada llamada recursiva debe acercar el problema hacia ese caso base.
- Si no se cumple esta regla (por ejemplo, si la función nunca alcanza un caso base), ocurrirá recursión infinita y el programa nunca terminará.

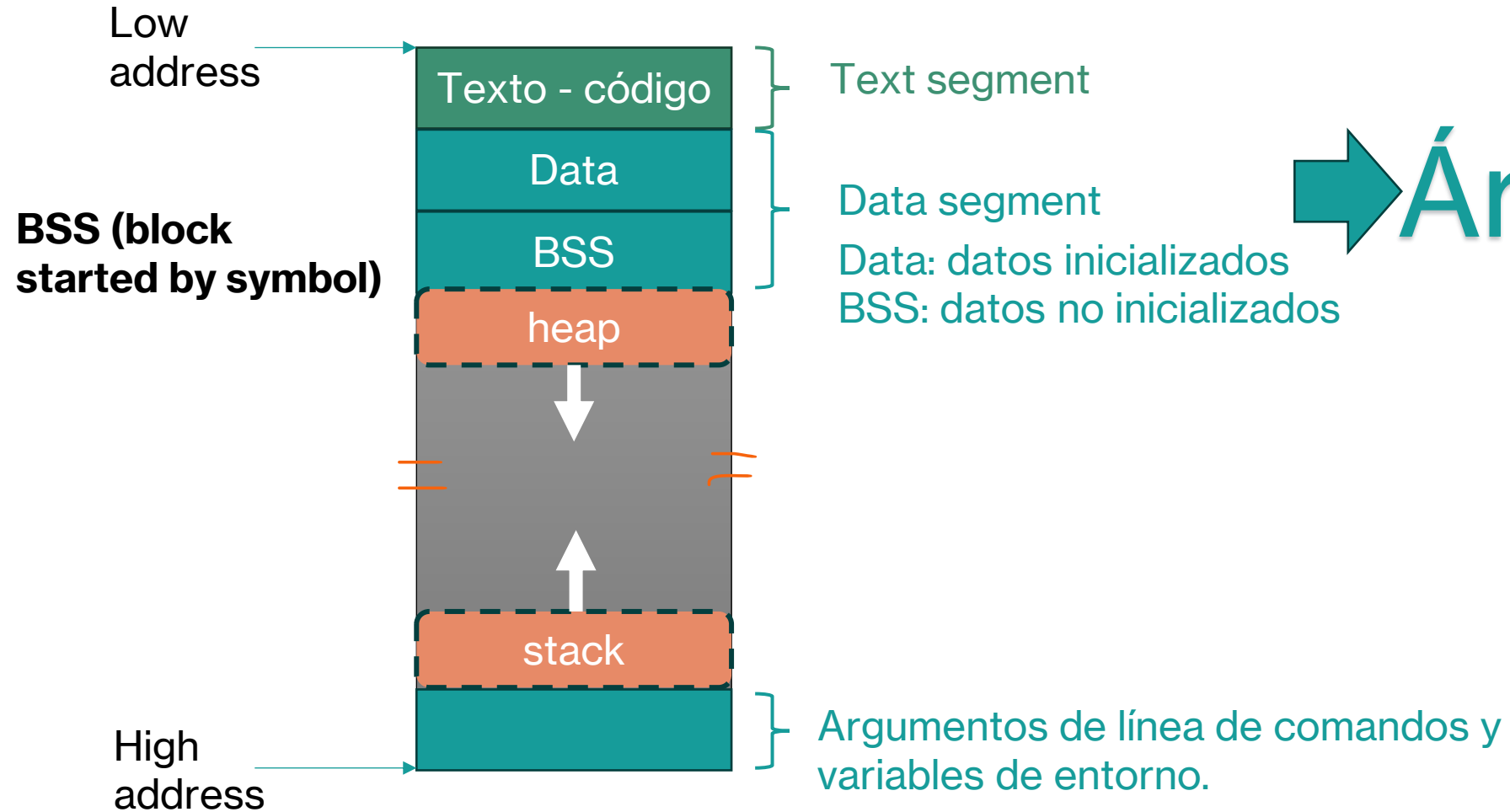


Recursividad vs Iteración

- A menudo, la recursión logra los mismos resultados a los de una iteración (bucle).
- Cualquier algoritmo recursivo puede expresarse de forma iterativa utilizando estructuras repetitivas.
- Sin embargo, la recursión ofrece un modo **declarativo** de pensar en el problema, donde describimos la solución en términos del propio problema a menor escala, lo que puede conducir a implementaciones más claras o concisas para ciertas tareas (hoy vamos a ver algunos ejemplos).
- No obstante, hay que recordar que la computadora ejecuta las llamadas recursivas de manera fundamentalmente iterativa, apoyándose en el stack de llamadas a función, como veremos a continuación.

El stack (REPASO)

Un programa en C, al ejecutarse, divide su espacio de memoria en **segmentos** bien definidos.



➡ Área Estática

El stack (REPASO)

La **pila** del programa es una estructura **LIFO** (Last In, First Out), típicamente ubicada en las partes más altas de la memoria. En la arquitectura estándar de computadoras PC x86, crece hacia la dirección cero.

Cuando se llama a una función se le asigna espacio en la **pila** para sus **variables** de **duración automáticas**.

Así es como las funciones recursivas en C pueden funcionar. Cada vez que una función recursiva se llama a sí misma, se usa un nuevo stack frame, por lo que un conjunto de variables no interfiere con las variables de otra instancia de la función.

El stack con funciones recursivas

```
int fr(int);

int main(void) ★
{
    int i;
    i=fr(3);
    printf("\ni=%d \n",i);
    return 0;
}

int fr(int i)
{
    if (i!=6) {
        printf("\ni=%d",i);
        i=fr(i+1);
    }
    return i;
}
```

i = *basura*
[control de pila]

➡ Tope de pila (SP)

El stack con funciones recursivas

```
int fr(int);

int main(void)
{
    int i;
    i=fr(3); 
    printf("\ni=%d \n",i);
    return 0;
}

int fr(int i)
{
    if (i!=6) {
        printf("\ni=%d",i);
        i=fr(i+1);
    }
    return i;
}
```

i = *basura*
[control de pila]

➡ Tope de pila (SP)

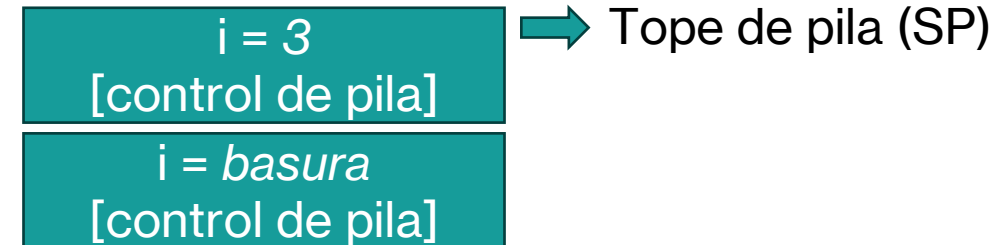
El stack con funciones recursivas

```
int fr(int);

int main(void)
{
    int i;
    i=fr(3);
    printf("\ni=%d \n",i);
    return 0;
}
```

```
//consola
i=3
```

```
int fr(int i) ★
{
    if (i!=6) {
        printf("\ni=%d",i);
        i=fr(i+1);
    }
    return i;
}
```



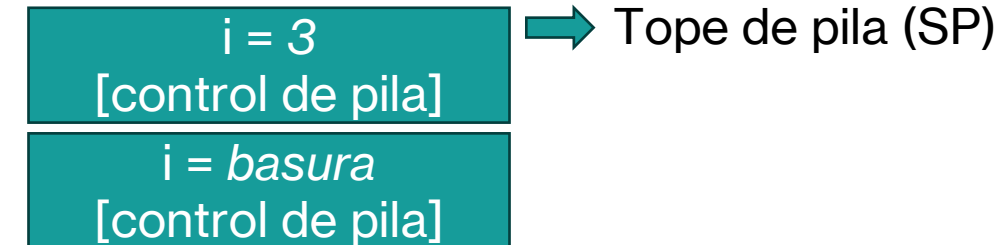
El stack con funciones recursivas

```
int fr(int);

int main(void)
{
    int i;
    i=fr(3);
    printf("\ni=%d \n",i);
    return 0;
}

int fr(int i)
{
    if (i!=6) {
        printf("\ni=%d",i);
        i=fr(i+1);
    }
    return i;
}
```

```
//consola
i=3
```



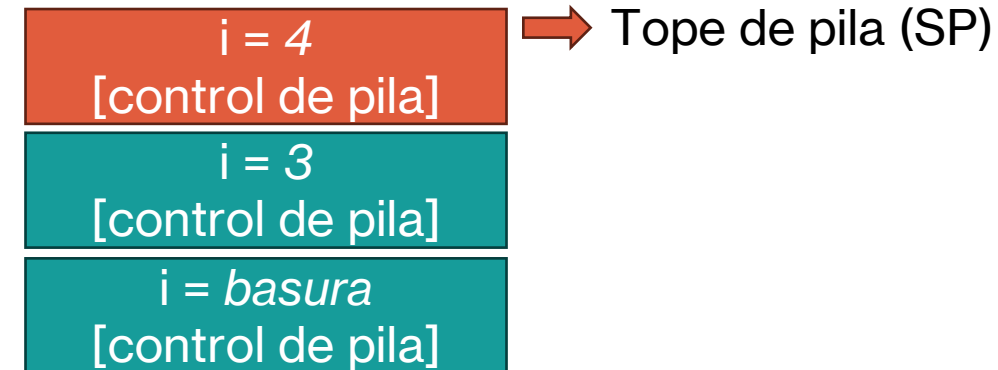
El stack con funciones recursivas

```
int fr(int);

int main(void)
{
    int i;
    i=fr(3);
    printf("\ni=%d \n",i);
    return 0;
}
```

```
int fr(int i) ★
{
    if (i!=6) {
        printf("\ni=%d",i);
        i=fr(i+1);
    }
    return i;
}
```

```
//consola
i=3
i=4
```



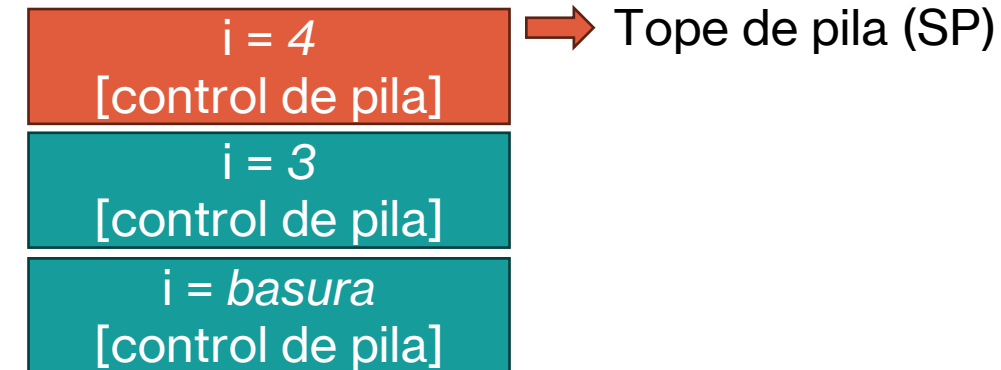
El stack con funciones recursivas

```
int fr(int);

int main(void)
{
    int i;
    i=fr(3);
    printf("\ni=%d \n",i);
    return 0;
}

int fr(int i)
{
    if (i!=6) {
        printf("\ni=%d",i);
        i=fr(i+1);
    }
    return i;
}
```

```
//consola
i=3
i=4
```



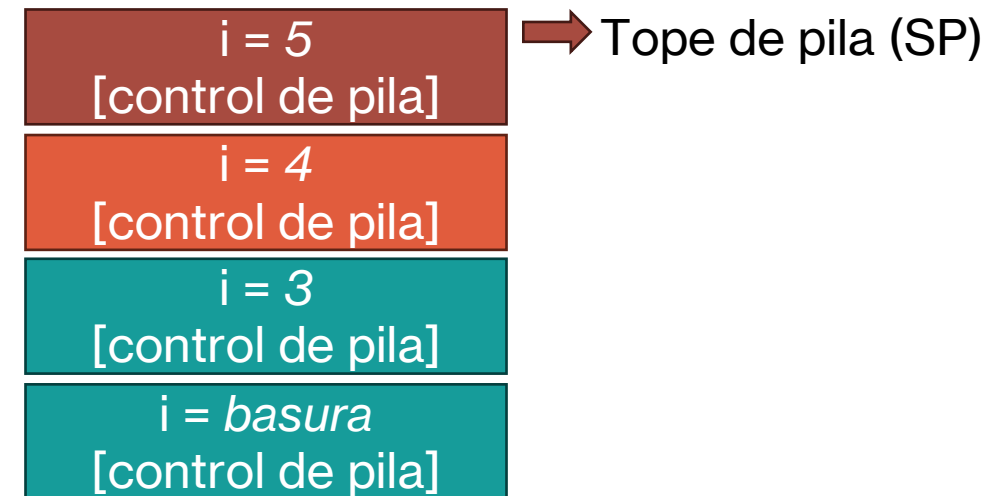
El stack con funciones recursivas

```
int fr(int);

int main(void)
{
    int i;
    i=fr(3);
    printf("\ni=%d \n",i);
    return 0;
}
```

```
int fr(int i) ★
{
    if (i!=6) {
        printf("\ni=%d",i);
        i=fr(i+1);
    }
    return i;
}
```

```
//consola
i=3
i=4
i=5
```



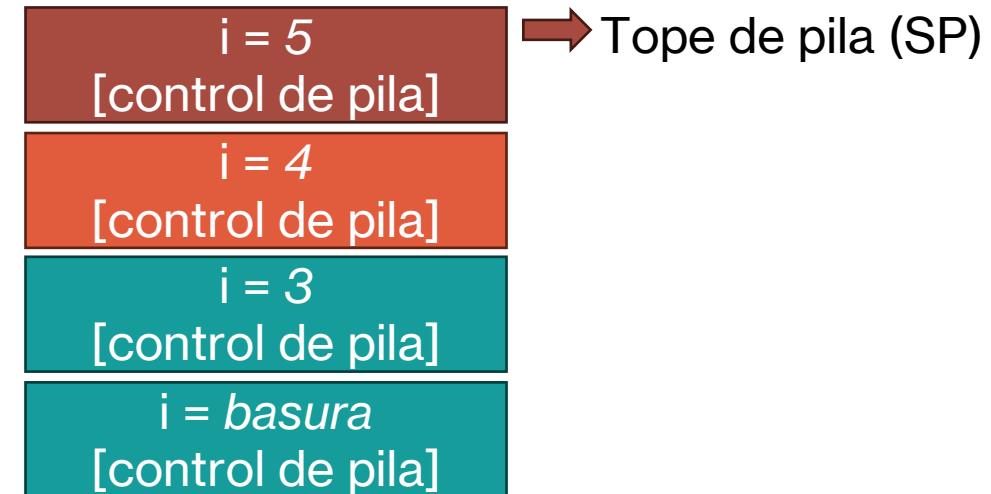
El stack con funciones recursivas

```
int fr(int);

int main(void)
{
    int i;
    i=fr(3);
    printf("\ni=%d \n",i);
    return 0;
}

int fr(int i)
{
    if (i!=6) {
        printf("\ni=%d",i);
        i=fr(i+1);
    }
    return i;
}
```

```
//consola
i=3
i=4
i=5
```



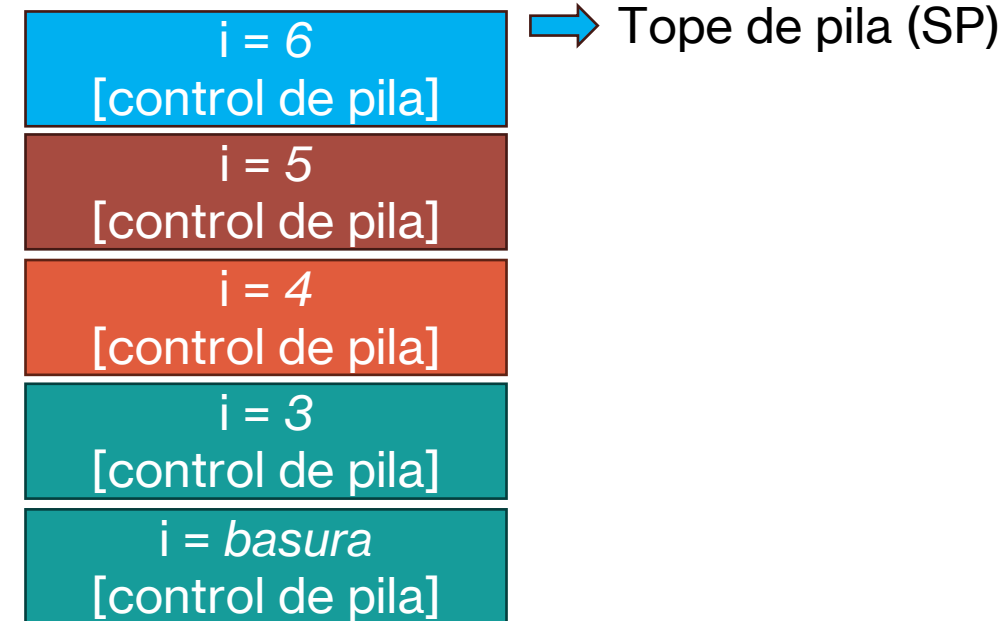
El stack con funciones recursivas

```
int fr(int);

int main(void)
{
    int i;
    i=fr(3);
    printf("\ni=%d \n",i);
    return 0;
}

int fr(int i) ★
{
    if (i!=6) {
        printf("\ni=%d",i);
        i=fr(i+1);
    }
    return i;
}
```

```
//consola
i=3
i=4
i=5
```



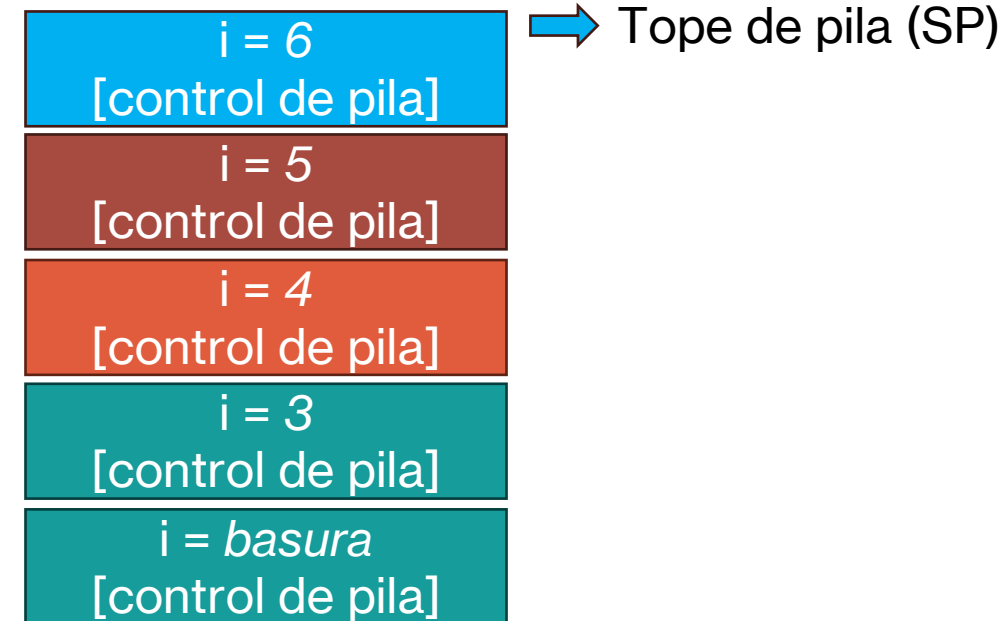
El stack con funciones recursivas

```
int fr(int);

int main(void)
{
    int i;
    i=fr(3);
    printf("\ni=%d \n",i);
    return 0;
}

int fr(int i)
{
    if (i!=6) {
        printf("\ni=%d",i);
        i=fr(i+1);
    }
    return i; ★
}
```

```
//consola
i=3
i=4
i=5
```



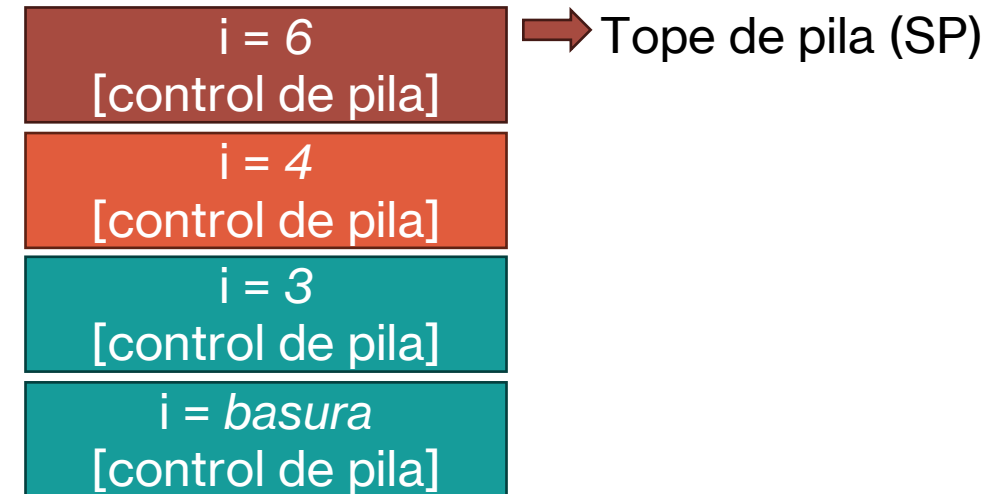
El stack con funciones recursivas

```
int fr(int);

int main(void)
{
    int i;
    i=fr(3);
    printf("\ni=%d \n",i);
    return 0;
}
```

```
int fr(int i)
{
    if (i!=6) {
        printf("\ni=%d",i);
        i=fr(i+1);
    }
    return i; ★
}
```

```
//consola
i=3
i=4
i=5
```



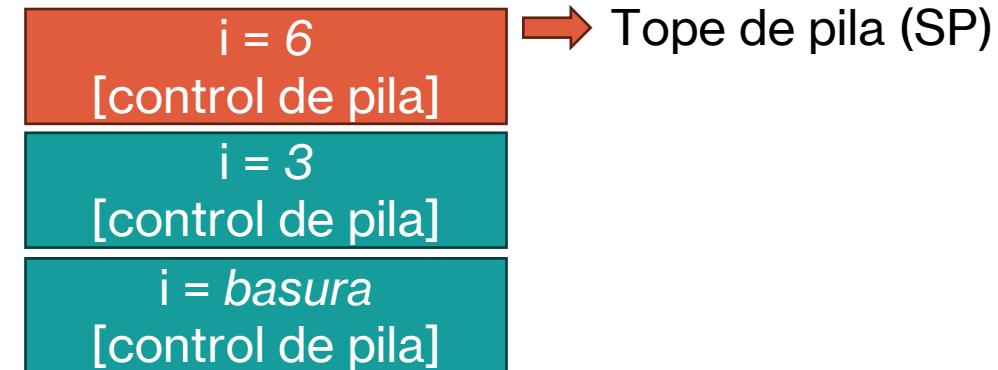
El stack con funciones recursivas

```
int fr(int);

int main(void)
{
    int i;
    i=fr(3);
    printf("\ni=%d \n",i);
    return 0;
}

int fr(int i)
{
    if (i!=6) {
        printf("\ni=%d",i);
        i=fr(i+1);
    }
    return i; ★
}
```

```
//consola
i=3
i=4
i=5
```



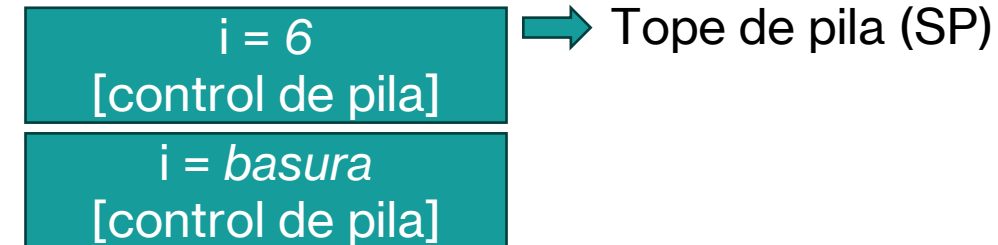
El stack con funciones recursivas

```
int fr(int);

int main(void)
{
    int i;
    i=fr(3);
    printf("\ni=%d \n",i);
    return 0;
}

int fr(int i)
{
    if (i!=6) {
        printf("\ni=%d",i);
        i=fr(i+1);
    }
    return i;★
}
```

```
//consola
i=3
i=4
i=5
```



El stack con funciones recursivas

```
int fr(int);

int main(void)
{
    int i;
    i=fr(3);
    ★ printf("\ni=%d \n",i);
    return 0;
}

int fr(int i)
{
    if (i!=6) {
        printf("\ni=%d",i);
        i=fr(i+1);
    }
    return i;
}
```

```
//consola
i=3
i=4
i=5
i=6
```

i = 6
[control de pila]

➡ Tope de pila (SP)

El stack con funciones recursivas

¿Qué pasa en este programa?

```
int fr(int);

int main(void)
{
    int i;
    i=fr(3);
    printf("\ni=%d \n",i);
    return 0;
}

int fr(int i)
{
    printf("\ni=%d",i);
    i=fr(i+1);
    return i;
}
```

1. Creamos un agujero de gusano



2. Tenemos un error en tiempo de ejecución: STACK OVERFLOW



Ejemplos típicos con resolución recursiva

EJEMPLO 01: Factorial de un numero

- El **factorial** de un número entero positivo n (denotado $n!$) se define matemáticamente como el producto de todos los enteros positivos desde 1 hasta n .
- Por definición, $0! = 1$.
- Por ejemplo, $5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$.
- Este es un caso clásico para demostrar la recursividad: la definición de factorial puede expresarse en términos del factorial de un número más pequeño.
- En forma recursiva:
 - Caso base: $0! = 1$ (el factorial de 0 es 1, sin necesidad de más cálculos).
 - Caso recursivo: para $n > 0 \rightarrow n! = n \times (n-1)!$
Es decir, el factorial de n se reduce al cálculo del factorial de $n-1$, y luego se multiplica por n .

Vamos al IDE

EJEMPLO 01: Factorial de un numero

Podemos representar la **traza de ejecución** de factorial(3) en una tabla para mayor claridad:

Llamada	Acción	Valor devuelto al terminar
factorial(3)	Calcula 3 * factorial(2)	<i>pendiente...</i>
factorial(2)	Calcula 2 * factorial(1)	<i>pendiente...</i>
factorial(1)	Calcula 1 * factorial(0)	<i>pendiente...</i>
factorial(0)	Caso base , retorna 1	1
factorial(1)	Sustituye factorial(0) por 1: retorna 1 * 1	1
factorial(2)	Sustituye factorial(1) por 1: retorna 2 * 1	2
factorial(3)	Sustituye factorial(2) por 2: retorna 3 * 2	6

En la traza anterior, observamos cómo las llamadas quedan **pendientes** (apiladas en el stack) hasta alcanzar el caso base. Luego, los resultados se van devolviendo y cada función pendiente puede terminar su cálculo. El valor final **factorial(3) = 6** se imprime en el **main**.

Este flujo demuestra claramente el uso de la pila: las llamadas se apilan y luego se despilan en orden inverso con los resultados.

EJEMPLO 02: Secuencia de Fibonacci

- La sucesión de **Fibonacci** es una serie clásica definida recursivamente.
- Comienza típicamente como **0, 1, 1, 2, 3, 5, 8, 13, 21, ...** donde **cada término (a partir del tercero) es la suma de los dos anteriores**.
- En muchas definiciones se considera $F(0) = 0$, $F(1) = 1$ como casos base, aunque algunas versiones empiezan la secuencia en 1,1.
 - Usaremos aquí la convención **$F(0)=0$, $F(1)=1$** .
- Matemáticamente:
 - **Caso base:** $F(0) = 0$ y $F(1) = 1$
 - **Caso recursivo:** $F(n) = F(n-1) + F(n-2)$ para $n \geq 2$. Es decir, el n -ésimo término se calcula sumando los dos términos anteriores.

Vamos al IDE

EJEMPLO 02: Secuencia de Fibonacci

- Cuando llamamos **fibonacci(n)** con n mayor o igual a 2, la función hará dos llamadas recursivas:
 - una para **fibonacci(n-1)** y
 - otra para **fibonacci(n-2)**,
sumando sus resultados.
- Por ejemplo, **fibonacci(5)** desencadena el cálculo de
 - **fibonacci(4)** y
 - **fibonacci(3)**;
a su vez
 - **fibonacci(4)** llama a
 - **fibonacci(3)** y
 - **fibonacci(2)**, etc.
- Esto forma un **árbol de llamadas** donde cada nodo (llamada) genera dos sub-llamadas (excepto los casos base). Los casos base **fibonacci(0)** y **fibonacci(1)** retornan inmediatamente 0 y 1, respectivamente, y esas respuestas se combinan hacia arriba en la suma.

EJEMPLO 02: Secuencia de Fibonacci

- El cálculo de `fibonacci(4)` sería:
 - **`fibonacci(4)`**
 - **`fibonacci(3)`**
 - **`fibonacci(2)`** 
 - **`fibonacci(1)` retorna 1 (caso base)**
 - **`fibonacci(0)` retorna 0 (caso base)**
 - **`fibonacci(2)` retorna $1 + 0 = 1$**
 - **`fibonacci(1)` retorna 1 (caso base)**
 - **`fibonacci(3)` retorna $1 + 1 = 2$**
 - **`fibonacci(2)`** 
 - **`fibonacci(1)` retorna 1**
 - **`fibonacci(0)` retorna 0**
 - **`fibonacci(2)` retorna $1 + 0 = 1$**
 - **`fibonacci(4)` retorna $\text{fibonacci}(3) + \text{fibonacci}(2) = 2 + 1 = 3$.**
- **0, 1, 1, 2, 3, 5, 8, 13, 21 $\rightarrow$ `fibonacci(4)` = 3 OK**
 - Notar que la implementación recursiva de Fibonacci hace cálculos repetidos.
 - Esto la vuelve ineficiente para n grandes.
 - Nos sirvió como ejemplo para explicar recursividad, pero no es la mejor solución a este problema.

EJEMPLO 02: Secuencia de Fibonacci

```
// Funcion recursiva que calcula el n-esimo termino Fibonacci
int fibonacci(int );

int main() {
    int n = 6;
    printf("El termino %d de Fibonacci es %d\n", n, fibonacci(n));
    return 0;
}

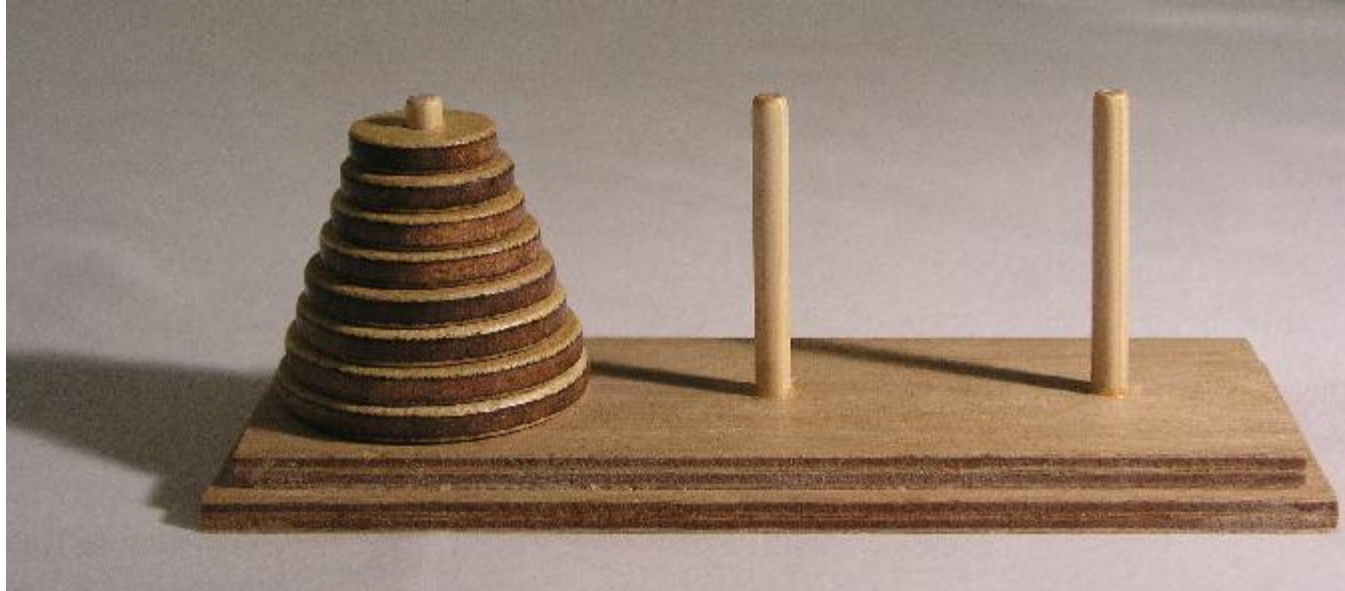
int fibonacci(int n) {
    if (n == 0) {
        return 0;                // Caso base 1: F(0) = 0
    } else if (n == 1) {
        return 1;                // Caso base 2: F(1) = 1
    } else {
        // Caso recursivo: F(n) = F(n-1) + F(n-2)
        return fibonacci(n - 1) + fibonacci(n - 2);
    }
}
```

Modificar el programa para imprimir la secuencia de Fibonacci hasta el termino 45 encontrado.
Comparar la eficiencia con un método iterativo.

EJEMPLO 03. Torres de Hanoi

Se cuenta una historia sobre un templo en la India, en Kashi Vishwanath, que contiene una gran sala con tres postes gastados por el tiempo, rodeados de 64 discos dorados. Los sacerdotes de Brahma, actuando bajo el mandato de una antigua profecía, comenzaron en un pasado remoto a mover estos discos según las reglas inmutables de Brahma, y no han cesado desde entonces.

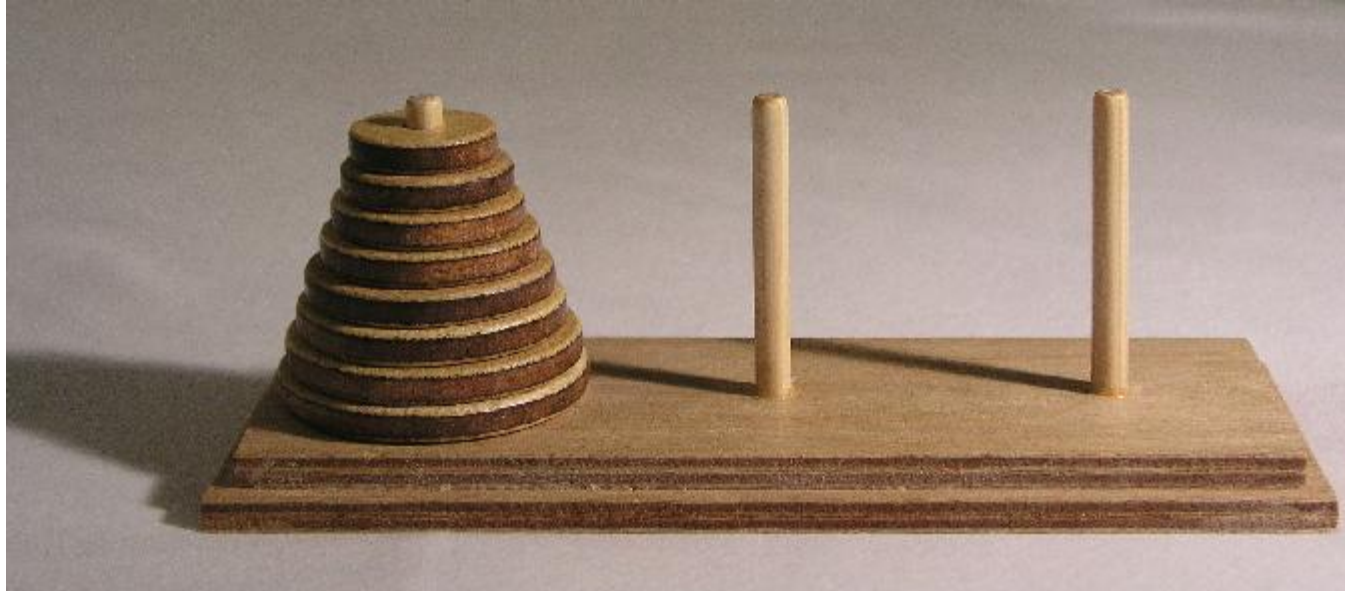
Según la leyenda, cuando se complete el último movimiento del rompecabezas, el mundo se terminará.



EJEMPLO 03. Torres de Hanoi

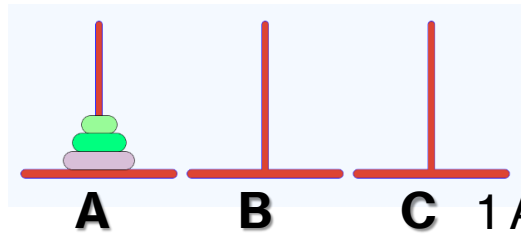
El enunciado consiste en lo siguiente:

- Tenemos tres postes y N discos apilados en uno de los postes (el poste de origen), ordenados de mayor (abajo) a menor (arriba).
- El objetivo es trasladar toda la pila de N discos desde el poste origen hasta otro poste destino, utilizando el tercer poste como auxiliar.
- Las **reglas** son:
 1. Solo se puede mover un disco a la vez, y
 2. Nunca se puede colocar un disco más grande encima de uno más pequeño.



<https://www.mathsisfun.com/games/towerofhanoi.html>

EJEMPLO 03. Torres de Hanoi

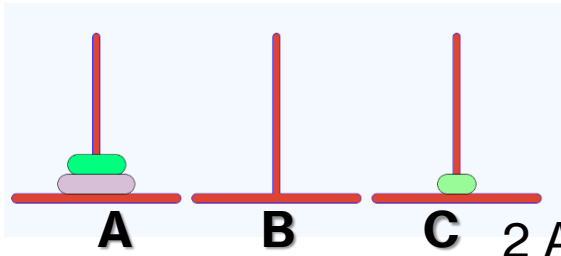


N=3

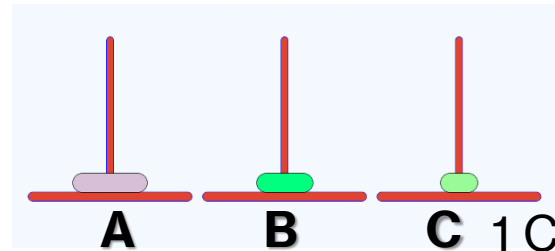
A → (B) → C

ORIGEN → (AUXILIAR) → DESTINO

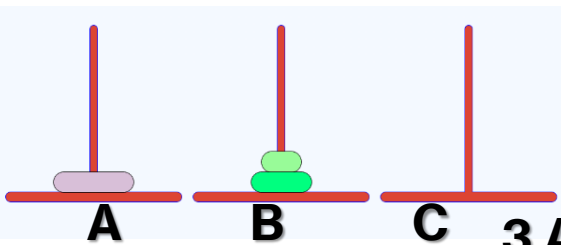
1 A → 1 C



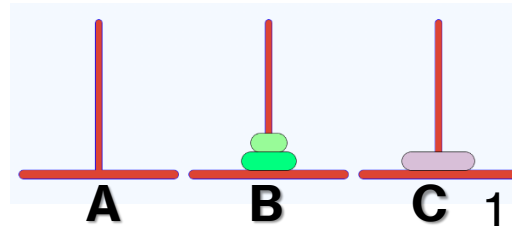
2 A → 2 B



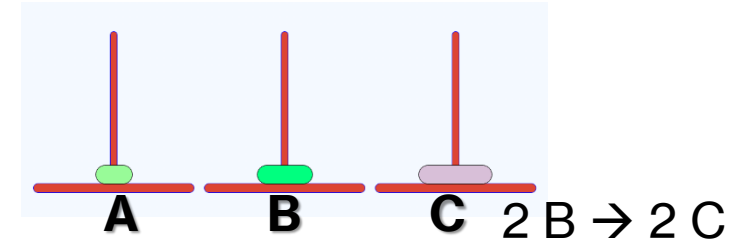
1 C → 1 B



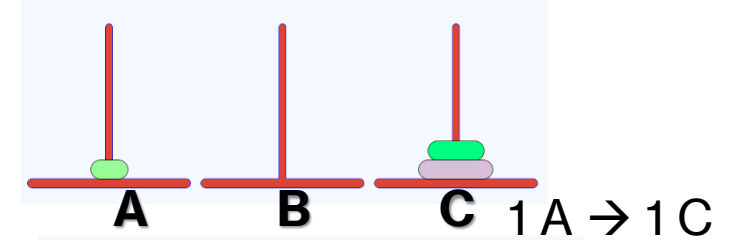
3 A → 3 C



1 B → 1 A

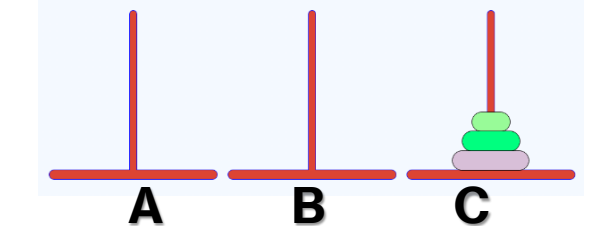


2 B → 2 C



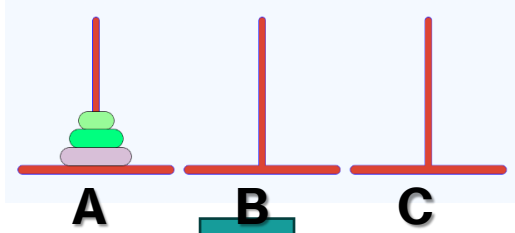
1 A → 1 C

Well Done !

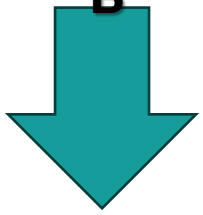


Busquemos la recursión

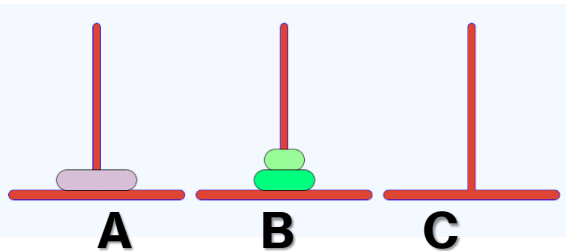
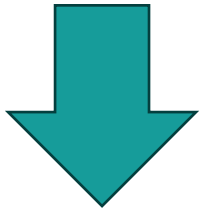
EJEMPLO 03. Torres de Hanoi



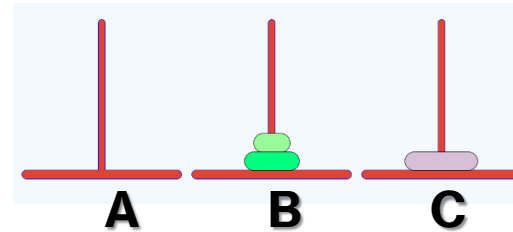
N=3
A → (B) → C



N=2
A → (C) → B



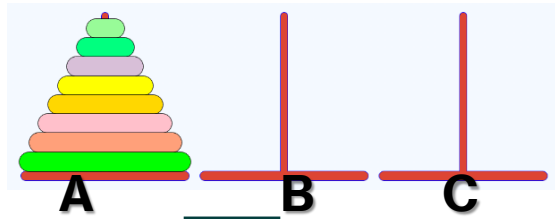
N=1
A → C



N=2
B → (A) → C



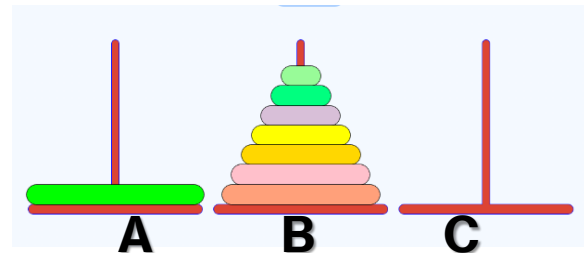
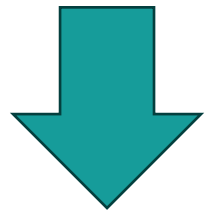
EJEMPLO 03. Torres de Hanoi



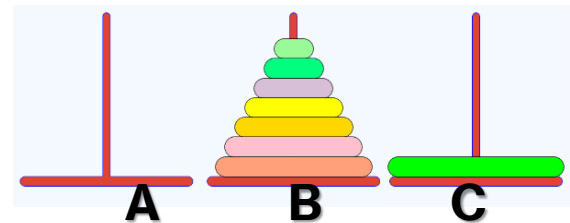
N
 $A \rightarrow (B) \rightarrow C$



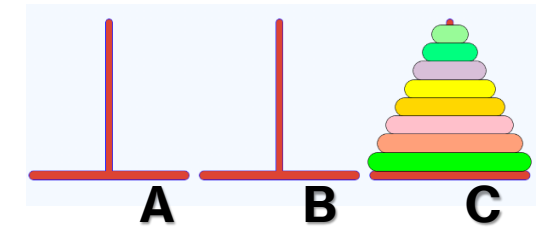
N-1
 $A \rightarrow (C) \rightarrow B$



N=1
 $A \rightarrow C$



N-1
 $B \rightarrow (A) \rightarrow C$



Vamos al IDE

```
hanoi (n , a , b , c ) {  
  
    if (n == 1) {  
        printf ("A → C");  
        return;  
    }  
  
    hanoi ( (n-1) , a , c , b);  
    printf ("A→C");  
    hanoi ((n-1), b , a , c);  
  
}
```

EJEMPLO 03. Torres de Hanoi

¿En cuanto tiempo van a terminar los monjes el problema con 100 discos?

- Con 3 discos se necesitan 7 movimientos
- Con 4 discos se necesitan 15 movimientos
- Con 5 discos se necesitan 31 movimientos
- Con N discos se necesitan $2^N - 1$ movimientos

Si imaginamos que los monjes son extremadamente rápidos y mueven de a 1 disco por segundo, tardaran:

$2^{64} - 1 = 18.446.744.073.709.551.615$ segundos

307.445.734.561.825.860,25 minutos

5.124.095.576.030.431 horas

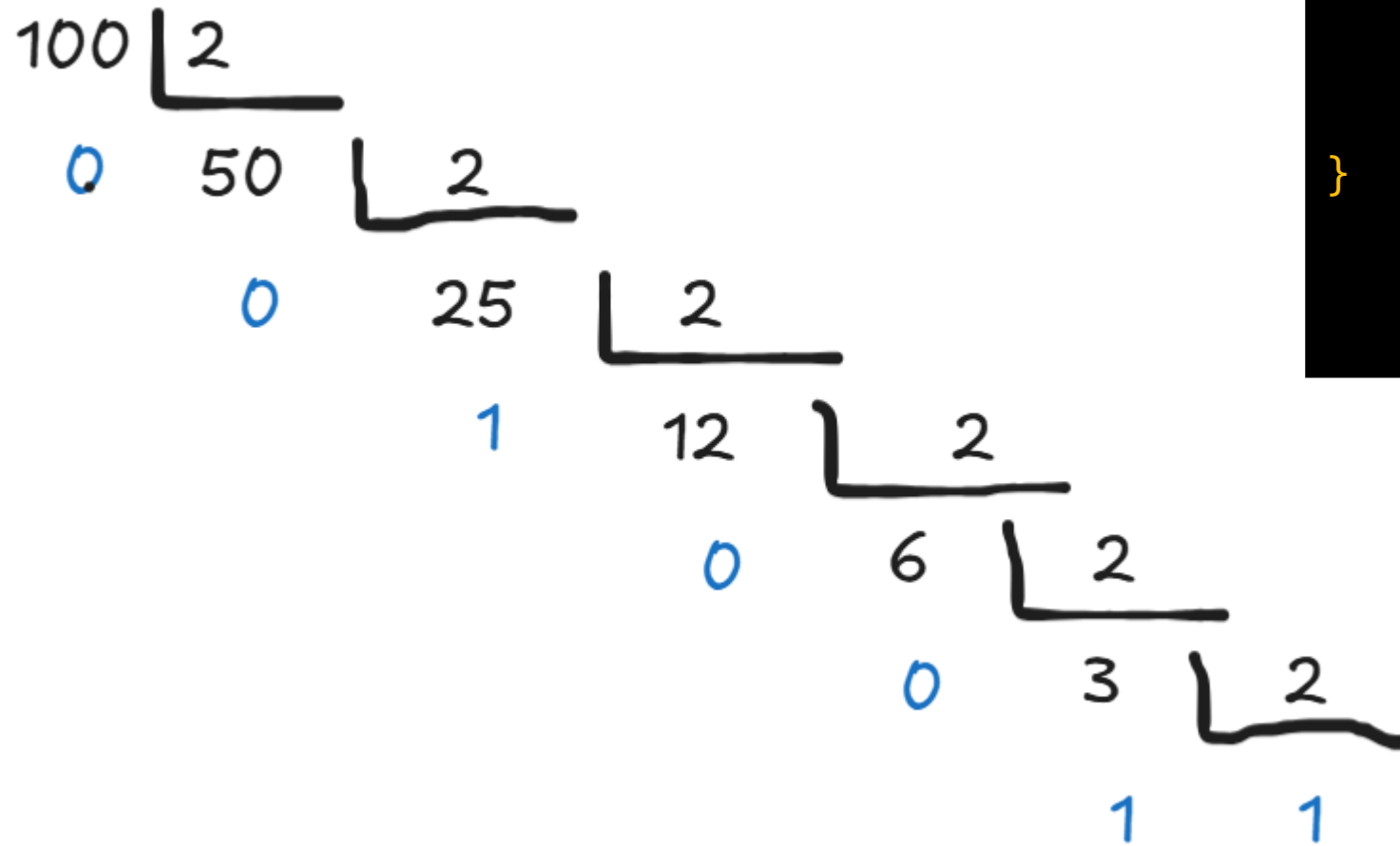
213.503.982.334.601,29 días

584.942.417.355,07 años

Estimación del tiempo de existencia del universo 13.800.000.000 años.



Ejemplo 04: Convertir de decimal a binario



```
void binario(int n) {  
    if (n == 0) return;  
    binario(n / 2);  
    printf("%d", n % 2);  
}
```

$$100_{10} = 1100100_2$$

Vamos al IDE

Ejemplo 05: Búsqueda Binaria

Partimos de un vector ordenado y una valor que estamos buscando: **objetivo = 11**

1	3	5	7	9	11	13
0	1	2	3	4	5	6

Búsqueda binaria entre **izquierda (0)** y **derecha (6)**:

$$medio = \frac{0 + 6}{2} = 3$$

¿Es **vec[medio] == objetivo**?

1. Si → Se termina la búsqueda
2. No → ¿Es **objetivo < vec [medio]**?
 1. Si → **Búsqueda binaria** entre **izquierda (0)** y **derecha (2)**
 2. No → **Búsqueda binaria** entre **izquierda (4)** y **derecha (6)**

Vamos al IDE

Búsqueda binaria entre **izquierda (4)** y **derecha (6)**

$$medio = \frac{4 + 6}{2} = 5$$

Empieza a quedar claro la recursión. Pero el caso base tiene dos partes:

1. Se encuentra el objetivo → **vec [medio] == objetivo → return medio;**
2. No se encuentra el objetivo → **izquierda > derecha → return -1;**

Conclusiones

- Los ejemplos que vimos son de **recursividad directa**, que es cuando **una función se llama a sí misma de forma explícita dentro de su propio cuerpo**.
- Sepan que también existe la **recursividad indirecta**, que ocurre cuando **una función A llama a una función B, que a su vez llama a la función A, generando un ciclo de llamadas recursivas entre funciones distintas**.
- La técnica que utilizamos en los problemas de **Torres de Hanoi** y **búsqueda binaria** se conoce como **"divide y vencerás"**, y es comúnmente utilizada junto con recursión. Consiste en **dividir el problema original en subproblemas más pequeños, resolverlos recursivamente y luego combinar sus soluciones**.
- Además, existe otra técnica llamada **backtracking** (o "vuelta atrás"), que se caracteriza por **explorar todas las posibles soluciones de un problema y retroceder (deshacer decisiones) cuando se detecta que una solución parcial no puede llevar a una solución válida**. Problemas clásicos que usan esta técnica son:
 - El problema de las **8 reinas**
 - El problema del salto del caballo

Ejercitación

1. De iterativo a recursivo: Aproximación de e

- Convierta la siguiente función iterativa en una versión **recursiva**.

La función calcula un valor aproximado de e, base de los logaritmos naturales, sumando la

serie: $\frac{1}{1!} + \frac{1}{2!} + \dots + \frac{1}{n!}$

```
double loge() {  
    double enl, delta, fact;  
    int n;  
    enl = fact = delta = 1.0;  
    n = 1;  
    do {  
        enl += delta;  
        n++;  
        fact *= n;  
        delta = 1.0 / fact;  
    } while (enl != enl + delta);  
    return enl;  
}
```

Ejercitación

2. Análisis de función recursiva: Factorial

Explique las razones por las cuales la siguiente función puede producir un resultado incorrecto al ejecutarse:

```
long factorial(long n) {  
    if (n == 0 || n == 1)  
        return 1;  
    else  
        return n * factorial(--n);  
}
```

3. Trazado de ejecución: Función recursiva f(n)

¿Qué valor retorna la llamada f(5)?
Mostrá la secuencia de llamadas recursivas generadas por el programa.

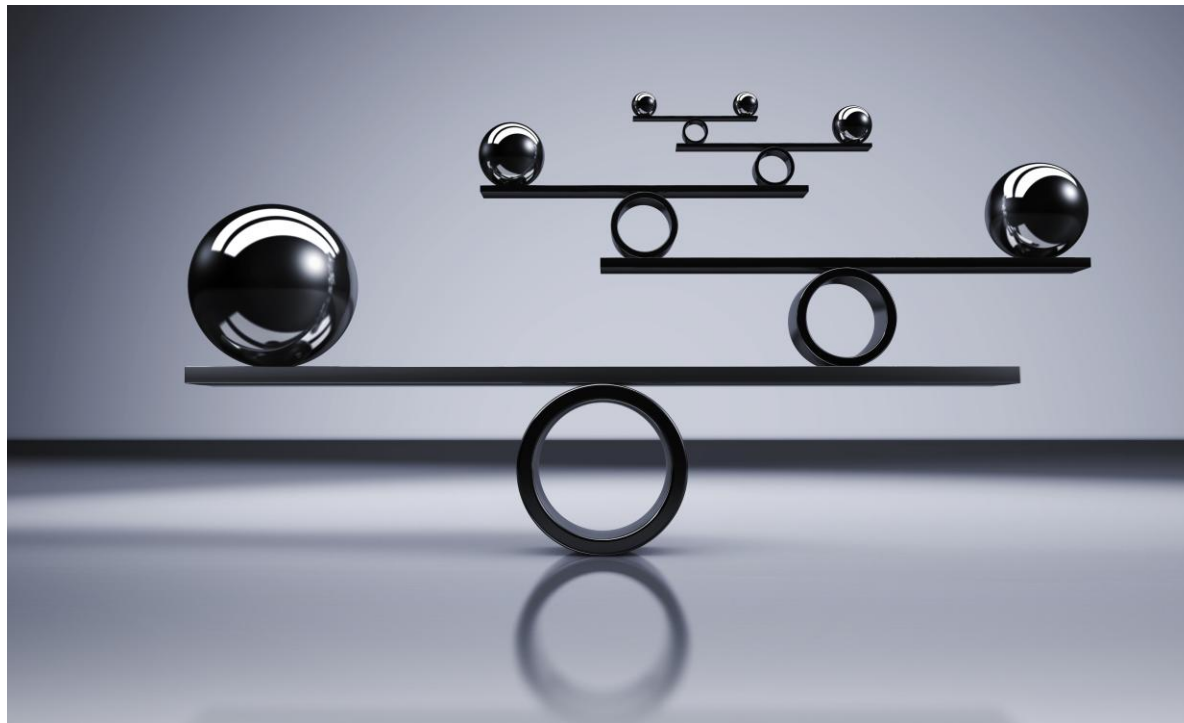
```
long f(int n) {  
    if (n == 0 || n == 1)  
        return 1;  
    else  
        return 3 * f(n - 2) + 2 * f(n - 1);  
}
```

Ejercitación

4. Recorrer strings: Contar vocales

Escriba una función recursiva de la forma → **int vocales(char *cd);**
que calcule y retorne la cantidad de vocales que tiene un string pasado por parámetro.

No se debe utilizar ninguna variable global ni recorrer el string con un bucle for o while.



A photograph of three shark fins protruding from the surface of the ocean. The largest fin is on the left, and two smaller ones are to its right. The water is dark blue with some white foam from a wake. The word "FIN" is written in white capital letters across the center of the image.

FIN